Model Diagnostic

The BF class can compute model diagnostics for a given model.

Lets consider the following model for a linear regression:

$$Y_i \sim \text{Normal}(\alpha + \beta X_i, \sigma)$$

$$\alpha \sim \text{Normal}(0, 1)$$

$$\beta \sim \text{Normal}(0, 1)$$

$$\sigma \sim \text{Uniform}(0, 50)$$

Python

```python
from BayesForge import bf
import jax.numpy as jnp

# setup platform-----------------------------------------------
m = bf(platform="cpu")

# import data -------------------------------------------------
data_path = m.load.howell1(only_path=True)
m.data(data_path, sep=";")
m.df = m.df[m.df.age > 18]
m.scale(data=["weight"])


# define model ------------------------------------------------
```

```python
def model(weight, height):
    a = m.dist.normal(178, 20, name="a")
    b = m.dist.log_normal(0, 1, name="b")
    s = m.dist.uniform(0, 50, name="s")
    m.dist.normal(a + b * weight, s, obs=height, shape=(weight.shape[0],))


# Run sampler ----------------------------------------------
m.fit(model, num_samples=500, num_chains=4)
m.summary()
```

```
/home/sosa/.local/lib/python3.13/site-packages/tqdm/auto.py:21: TqdmWarning:

IProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs

An NVIDIA GPU may be present on this machine, but a CUDA-enabled jaxlib is not installed. Fal

bf v 0.0.60 package loaded
jax.local_device_count 32

  0%|          | 0/1500 [00:00<?, ?it/s]Compiling.. :   0%|          | 0/1500 [00:00<?, ?it/s
  0%|          | 0/1500 [00:00<?, ?it/s]
Compiling.. :   0%|          | 0/1500 [00:00<?, ?it/s]

  0%|          | 0/1500 [00:00<?, ?it/s]

Compiling.. :   0%|          | 0/1500 [00:00<?, ?it/s]


  0%|          | 0/1500 [00:00<?, ?it/s]


Compiling.. :   0%|          | 0/1500 [00:00<?, ?it/s]Running chain 0:   0%|          | 0/150
Running chain 1:   0%|          | 0/1500 [00:01<?, ?it/s]

Running chain 2:   0%|          | 0/1500 [00:01<?, ?it/s]


Running chain 3:   0%|          | 0/1500 [00:01<?, ?it/s]Running chain 0: 100%|     | 1500/1
Running chain 2: 100%|     | 1500/1500 [00:01<00:00, 927.18it/s]
Running chain 1: 100%|     | 1500/1500 [00:01<00:00, 924.57it/s]
Running chain 3: 100%|     | 1500/1500 [00:01<00:00, 926.47it/s]
```

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcse_sd
a	154.64	0.27	154.21	155.06	1980.83	1634.51	1.0	0.01	0.01
b	5.82	0.26	5.41	6.24	1895.99	1406.63	1.0	0.01	0.01
s	5.14	0.20	4.84	5.47	1944.70	1379.35	1.0	0.00	0.00

R

```r
library(BayesForge)

# setup platform-----------------------------------------------
m <- importBF(platform = "cpu")

# import data -------------------------------------------------
data_path <- m$load$howell1(only_path = TRUE)
m$data(data_path, sep = ";")
m$df <- m$df[m$df$age > 18, ]
m$scale(list("weight"))
m$data_to_model(list("weight", "height"))

# define model ------------------------------------------------
model <- function(weight, height) {
    a <- bf.dist.normal(178, 20, name = "a")
    b <- bf.dist.log_normal(0, 1, name = "b")
    s <- bf.dist.uniform(0, 50, name = "s")
    bf.dist.normal(a + b * weight, s, obs = height)
}

# Run sampler -------------------------------------------------
m$fit(model, num_samples = 500, num_chains = 4)
m$summary()
```

Julia

```julia
using PyBayesForge

# setup platform-----------------------------------------------
m = importBF(platform = "cpu")

# import data -------------------------------------------------
```

```
data_path = m.load.howell1(only_path = true)
m.data(data_path, sep = ";")
m.df = m.df[m.df.age > 18]
m.scale(["weight"])

# define model ------------------------------------------------
@BF function model(weight, height)
    a = m.dist.normal(178, 20, name = "a")
    b = m.dist.log_normal(0, 1, name = "b")
    s = m.dist.uniform(0, 50, name = "s")
    m.dist.normal(a + b * weight, s, obs = height)
end

# Run sampler -------------------------------------------------
m.fit(model, num_samples = 500, num_chains = 4)
m.summary()
```

List of all available diagnostics

For additional documentation check the diagnostics API reference

Every plot takes `x_label=` and `y_label=` to replace the axis titles (`title=` for the plot title). A string sets the title of every panel; a list sets one per panel, in order (`None` keeps a panel's own title), for example `m.diag.ppc_hist(x_label="Height (cm)")` or `m.diag.posterior(x_label=["intercept", "slope", "sigma"])`.

Predictions from model based on specific data value

Python

```
m.sample() # Predictions from model base on data in data_on_model
m.sample(data=dict(weight=jnp.array([0.4])), remove_obs=False)# Predictions from a given valu
```

```
{'height': Array([[155.08251204],
        [152.4286963 ],
        [150.36576259],
        ...,
        [155.53146424],
        [153.53308362],
        [163.6036096 ]], dtype=float64)}
```

R

```
m$sample() # Predictions from model base on data in data_on_model
m$sample(data = list(weight = jnp$array(c(0.4))), remove_obs = FALSE) # Predictions from a gi
```

Julia

```
m.sample() # Predictions from model base on data in data_on_model
m.sample(data = pydict(weight = jnp.array([0.4])), remove_obs = false) # Predictions from a g
```

Forest plot of estimated values

Python

```
m.diag.forest()
```

```
Unable to display output for mime type(s): text/html
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$forest()
```

Julia

```
@pyplot m.diag.forest()
```

Density plots of the posterior distribution

Python

```
m.diag.density()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$density()
```

Julia

```
@pyplot m.diag.density()
```

Posterior distribution plots

Python

```
m.diag.posterior()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$posterior()
```

Julia

```
@pyplot m.diag.posterior()
```

Trace plots for MCMC chains

Python

```
m.diag.plot_trace()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$plot_trace()
```

Julia

```
@pyplot m.diag.plot_trace()
```

Pairwise plots of the posterior distribution

Python

```
m.diag.pair()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$pair()
```

Julia

```
@pyplot m.diag.pair()
```

Plot autocorrelation of MCMC chains

Python

```
m.diag.autocor()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$autocor()
```

Julia

```
@pyplot m.diag.autocor()
```

Create rank plots for MCMC chains

Python

```
m.diag.rank()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$rank()
```

Julia

```
@pyplot m.diag.rank()
```

Evolution of effective sample size across iterations

Python

```
m.diag.plot_ess()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$plot_ess()
```

Julia

```
@pyplot m.diag.plot_ess()
```

HMC diagnostics: divergences and energy

Check this plot first: if the sampler failed, the other diagnostics are not meaningful.

- **Top row, one panel per chain:** the distribution of the energy E around its mean (blue, *marginal energy*) and of its change between two successive iterations, ΔE (orange, *energy transition*). When the two histograms have about the same width, the sampler can cross the whole posterior from one iteration to the next. When ΔE is much narrower than E, it explores the tails badly (typical of a hierarchical "funnel"). The panel title gives E-BFMI $= \Sigma\ \Delta E^2\ /\ \Sigma\ (E - \bar{E})^2$; values below 0.3 are flagged with .
- **Bottom row:** the number of divergent transitions per chain (ideal: 0). A divergence means that the integrator failed in a region of strong curvature, which is then under-explored. With a few divergences, raise `target_accept` (0.95–0.99); with many, reparametrise (for example a non-centred random effect).

Python

```
m.diag.divergence_energy()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$divergence_energy()
```

Julia

```
@pyplot m.diag.divergence_energy()
```

Posterior predictive checks

All posterior predictive checks compare the observed outcome with *predictions*: datasets of the outcome simulated by the fitted model, one per posterior draw (`m.diag.get_yrep()`). Both are taken from the fitted model automatically, and the plots are labelled with the name of the observed site (here `height`, from `obs=height`). With several observed sites, each function returns one figure per site; pass `site=` to choose one. The idea is always the same: if the model is good, the data it simulates should look like the real data. For count or categorical outcomes, `m.diag.ppc_bars()` and `m.diag.ppc_rootogram()` instead compare observed counts with predicted counts per category.

Density overlay

Overlays the density of the observed data (black) with the densities of n predicted datasets (blue). The black curve should look like one more blue curve; a different shape (two modes, skewness, mass below 0 for a positive outcome) points to a missing group or a wrong likelihood.

Python

```
m.diag.ppc_density()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$ppc_density()
```

Julia

```
@pyplot m.diag.ppc_density()
```

Histogram small multiples

The histogram of the observed data (first panel, black) next to the histograms of n predicted datasets ("Prediction 1", "Prediction 2", …). The observed histogram should not stand out among the predictions; look at the number of modes, the skewness, the range and the number of zeros.

Python

```
m.diag.ppc_hist()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$ppc_hist()
```

Julia

```
@pyplot m.diag.ppc_hist()
```

Boxplots

The box of the observed data (left, black) next to the boxes of n predicted datasets ("pred 1", "pred 2", …): median, quartiles, whiskers and extreme points. The observed box should look like the others; an observed box that is systematically higher, wider or narrower, or extreme values that the predictions never produce (tails too light: try a Student-t likelihood), signal misfit.

Python

```
m.diag.ppc_boxplot()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$ppc_boxplot()
```

Julia

```
@pyplot m.diag.ppc_boxplot()
```

Test statistic

Each predicted dataset is summarised by one number T (the curve is the distribution of T over the predictions) and T of the observed data is marked by the vertical line. The title gives the Bayesian p-value P(T(predicted) T(observed)) and its two-sided version: around 0.5 is fine, a two-sided value below 0.05 means the model does not reproduce this aspect of the data.

Choose the statistic with `stat=`: `"mean"` (default), `"median"`, `"sd"`, `"var"`, `"min"`, `"max"`, `"q25"`, `"q75"`, `"skew"`, `"n_zero"`, `"n_pos"`, or (Python) any function of a 1D array, for example `stat=lambda v: np.percentile(v, 90)`. The mean is almost always reproduced because the model has a parameter for it; statistics the model does not fit directly (`"sd"` for over-dispersion, `"max"`/`"min"` for the tails, `"skew"`, `"n_zero"` for counts) are more informative.

Python

```
m.diag.ppc_stat(stat="sd")
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$ppc_stat(stat = "sd")
```

Julia

```
@pyplot m.diag.ppc_stat(stat = "sd")
```

Predictive intervals

One vertical bar per observation: the thin line is the 90% predictive interval (`prob_outer`), the thick line the 50% interval (`prob`), the white point the median prediction and the black point the observed value. About 50% of the black points should fall on the thick bars and 90% on the thin ones. Many points outside mean the model is over-confident; all points close to the median mean it is too cautious. Pass `x=` (a predictor) to see *where* the model misses.

Python

```
m.diag.ppc_intervals()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$ppc_intervals()
```

Julia

```
@pyplot m.diag.ppc_intervals()
```

Predictive ribbon

The same intervals drawn as continuous ribbons (light: 90%, dark: 50%; line: median prediction) with the observed values as points. Easier to read than the bars when `x` is a continuous variable, here the predictor `weight`: the ribbon should follow the cloud of points over the whole range of x, without a curvature it misses.

Python

```
m.diag.ppc_ribbon(x=m.df.weight)
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$ppc_ribbon(x = m$df$weight)
```

Julia

```
@pyplot m.diag.ppc_ribbon(x = m.df.weight)
```

Observed versus mean prediction

Observed value (y axis) against the mean prediction of each observation (x axis); the dashed diagonal is a perfect prediction. A cloud of constant width around the diagonal is good. A funnel (spread growing with the prediction) indicates heteroscedasticity, a curve an unmodelled non-linearity, and a slope below 1 shrinkage (expected in hierarchical models, suspicious when strong).

Python

```
m.diag.ppc_error_scatter()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$ppc_error_scatter()
```

Julia

```
@pyplot m.diag.ppc_error_scatter()
```

Residuals

Histogram of the residuals r_i = observed_i − mean prediction_i. It should be centred on 0 and roughly symmetric; a shift is a systematic bias, skewness a wrong likelihood family, two modes a missing group, and long tails a few badly predicted observations (see LOO influence).

Python

```
m.diag.ppc_error_hist()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$ppc_error_hist()
```

Julia

```
@pyplot m.diag.ppc_error_hist()
```

Observed versus individual predictions

Each panel plots the observed data against **one** predicted dataset (not the mean), so the observation noise is included and the cloud is wider than in the plot above. The width of the cloud around the diagonal should be similar from one panel to the next; observed values that systematically overshoot on one side mean the simulated noise is mis-calibrated.

Python

```
m.diag.ppc_scatter()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$ppc_scatter()
```

Julia

```
@pyplot m.diag.ppc_scatter()
```

LOO-PIT uniformity check

For each observation, the PIT (probability integral transform) is the quantile of its predictive distribution at which the observed value falls: PIT_i = P(prediction_i y_i). If the model is calibrated these values are uniform between 0 and 1, so their density (blue) is flat around the dashed line at 1. The grey curves are densities of the same number of truly uniform values: they show how far from flat a perfectly calibrated model can be by chance, so a blue curve that stays inside the grey band is fine.

Shape of the blue curve	Diagnosis
flat, inside the grey band	calibrated
U-shaped (high at 0 and 1)	predictions too narrow: the model is over-confident
bell-shaped (peak at 0.5)	predictions too wide: the model is too cautious
tilted to one side	bias: the model over- or under-predicts

PIT versus LOO-PIT. The plain PIT scores each observation with draws from a posterior that was fitted *to that same observation*, so the predictions are pulled towards it and the PIT values towards 0.5: a model that is over-confident can look calibrated. The LOO-PIT uses the leave-one-out predictive distribution instead, each observation being scored by a posterior fitted without it. Refitting n times is not needed: Pareto-smoothed importance sampling reweights the draws with the pointwise log-likelihood. `m.diag.ppc_loo_pit()` computes that log-likelihood from the fitted model automatically. When you pass your own predictions with `yrep=`, also pass `log_likelihood=` with the pointwise log-likelihood of the same draws, shape (draws, observations); without it the plain PIT is plotted and a warning is raised.

Python

```
m.diag.ppc_loo_pit()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$ppc_loo_pit()
```

Julia

```
@pyplot m.diag.ppc_loo_pit()
```

Predictive intervals sorted by observed value

The predictive intervals with the observations ordered by their observed value, so the black points rise from left to right. The medians should follow the black points. A little "regression towards the mean" is normal (the smallest values slightly over-predicted, the largest slightly under-predicted, because the posterior shrinks them); a strong and systematic gap means the model flattens the extremes, for example because a predictor or a variance term is missing.

Python

```
m.diag.ppc_intervals_sorted()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$ppc_intervals_sorted()
```

Julia

```
@pyplot m.diag.ppc_intervals_sorted()
```

Regression plot

Observed data plus n smooth posterior regression lines E[y | x, theta] and their posterior mean, overlaid against a predictor variable.

Python

```
m.diag.plot_regression("weight", n = 20)
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$plot_regression("weight", n = 20L)
```

Julia

```
@pyplot m.diag.plot_regression("weight", n = 20)
```

Sensitivity

Prior sensitivity

Overlays the posteriors from models fitted with different priors to assess how much the conclusions change when the priors vary.

Python

```
# Refit the same model with a tighter prior on b
posteriors_weak = m.posteriors

def model_strong(weight, height):
    a = m.dist.normal(178, 20, name="a")
    b = m.dist.log_normal(0, 0.1, name="b")
    s = m.dist.uniform(0, 50, name="s")
    m.dist.normal(a + b * weight, s, obs=height, shape=(weight.shape[0],))

m.fit(model_strong, num_samples=500, num_chains=4)

m.diag.prior_sensitivity([posteriors_weak, m.posteriors],
                         labels=["weak prior", "strong prior"])
```

```
  0%|          | 0/1500 [00:00<?, ?it/s]Compiling.. :   0%|          | 0/1500 [00:00<?, ?it/s
  0%|          | 0/1500 [00:00<?, ?it/s]
Compiling.. :   0%|          | 0/1500 [00:00<?, ?it/s]

  0%|          | 0/1500 [00:00<?, ?it/s]

Compiling.. :   0%|          | 0/1500 [00:00<?, ?it/s]


  0%|          | 0/1500 [00:00<?, ?it/s]


Compiling.. :   0%|          | 0/1500 [00:00<?, ?it/s]Running chain 0:   0%|          | 0/150
Running chain 1:   0%|          | 0/1500 [00:01<?, ?it/s]

Running chain 2:   0%|          | 0/1500 [00:01<?, ?it/s]


Running chain 3:   0%|          | 0/1500 [00:01<?, ?it/s]Running chain 3: 100%|     | 1500/1

Running chain 1:  55%|       | 825/1500 [00:01<00:00, 8173.68it/s]Running chain 0:  60%|

Running chain 2:  50%|        | 750/1500 [00:01<00:00, 7040.38it/s]Running chain 0: 100%|
Running chain 1: 100%|     | 1500/1500 [00:01<00:00, 1243.52it/s]
Running chain 2: 100%|     | 1500/1500 [00:01<00:00, 1247.17it/s]

Unable to display output for mime type(s): text/html
```

R

```r
# Refit the same model with a tighter prior on b
posteriors_weak <- m$posteriors

model_strong <- function(weight, height) {
    a <- bf.dist.normal(178, 20, name = "a")
    b <- bf.dist.log_normal(0, 0.1, name = "b")
    s <- bf.dist.uniform(0, 50, name = "s")
    bf.dist.normal(a + b * weight, s, obs = height)
}

m$fit(model_strong, num_samples = 500, num_chains = 4)
```

```
m$diag$prior_sensitivity(list(posteriors_weak, m$posteriors),
                         labels = list("weak prior", "strong prior"))
```

Julia

```
# Refit the same model with a tighter prior on b
posteriors_weak = m.posteriors

@BF function model_strong(weight, height)
    a = m.dist.normal(178, 20, name = "a")
    b = m.dist.log_normal(0, 0.1, name = "b")
    s = m.dist.uniform(0, 50, name = "s")
    m.dist.normal(a + b * weight, s, obs = height)
end

m.fit(model_strong, num_samples = 500, num_chains = 4)

@pyplot m.diag.prior_sensitivity([posteriors_weak, m.posteriors],
                                 labels = ["weak prior", "strong prior"])
```

LOO influence

One point per observation with its Pareto k, from PSIS-LOO: how much the posterior would change if this observation were left out.

k	Reading
< 0.5	fine
$0.5 – 0.7$ (orange line)	noticeable influence, the LOO estimate is still reliable
> 0.7 (red line, red points)	very influential observation; LOO is unreliable for it

A few red points are often outliers or data errors (hover to see their index and value). Many red points mean the model is misspecified or too flexible (for example one parameter per observation); try a robust likelihood (Student-t) or a more structured model. When some k exceed 0.7, model comparison with `m.diag.loo()` / `compare` is not reliable.

Python

```
m.diag.influence()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$influence()
```

Julia

```
@pyplot m.diag.influence()
```

Calibration

For several interval levels (50%, 60%, …, 99%), the share of observed values that actually fall inside the predictive interval of that level (y axis) against the level itself (x axis); the dashed diagonal is perfect calibration. Points below the diagonal (the "90%" interval contains only, say, 70% of the data) mean the model is over-confident, which is the most serious case; points above it mean the model is too cautious. These intervals are not leave-one-out, so a slight over-coverage is expected.

Python

```
m.diag.calibration()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$calibration()
```

Julia

```
@pyplot m.diag.calibration()
```

Multimodality check

Flags parameters whose posterior density has more than one local maximum (potential multimodality) and overlays the densities of the individual chains.

Python

```
m.diag.multimodality()
```

```
Unable to display output for mime type(s): text/html
```

R

```
m$diag$multimodality()
```

Julia

```
@pyplot m.diag.multimodality()
```

Pareto-smoothed

Python

```
m.diag.loo()
```

```
Computed from 2000 posterior samples and 346 observations log-likelihood matrix.

         Estimate       SE
elpd_loo -1140.60    11.94
p_loo        2.45        -
```

R

```
m$diag$loo()
```

Julia

```
m.diag.loo()
```

Widely applicable information criterion

Python

```
m.diag.WAIC()
```

```
elpd_waic       -1140.589533
se                 11.91806
p_waic             2.440741
n_samples              2000
n_data_points           346
scale                   log
dtype: object
```

R

```
m$diag$WAIC()
```

Julia

```
m.diag.WAIC()
```